

CodeMap: Language Support & Code Parsing Architecture

Overview

CodeMap is designed to analyze source code and generate detailed control-flow graphs and dependency maps. To achieve this across multiple programming languages, the backend relies on abstract syntax trees (ASTs) generated by the high-performance parsing library, **Tree-sitter**.

This document details the exact process of how source code is transformed from raw text into JSON graph objects that the React frontend renders, focusing on how language support and cyclomatic complexity scoring are implemented.

1. The Core Parsing Engine

The backend (`main_v2.py`) is a FastAPI service that receives raw source code and a target language via a REST API endpoint (`/analyze`).

Instead of relying on regex or basic string matching, CodeMap uses **Tree-sitter**. Tree-sitter creates a concrete syntax tree of the code, allowing the backend to traverse structural nodes (like `if_statement`, `for_statement`, `call_expression`, `method_declaration`).

Supported Languages

CodeMap dynamically selects the Tree-sitter grammar based on the file extension selected in the frontend:

- **Python** (`.py`)
- **JavaScript** (`.js`, `.jsx`)
- **TypeScript** (`.ts`, `.tsx`)
- **Java** (`.java`)
- **C/C++** (`.c`, `.cpp`, `.h`)

2. Pipeline Execution Steps

When a request hits the `/analyze` endpoint:

1. **Language Routing:** The `/analyze` endpoint delegates the source code to a specific handler function:

- `parse_python_code()`
- `parse_java_structure()`
- `parse_js_structure()`
- `parse_cpp_structure()`

2. **Graph Initialization:** A `ReactFlowBuilder` class is instantiated. This class manages an internal state of nodes and edges, providing helper methods like `add_node()` and `add_edge()` that format the data precisely as the `reactflow` library expects (`id`, `data.label`, `type`, `position`).
3. **AST Traversal:** The tree-sitter parser parses the raw string into a `Tree`. The root node's children are iterated over:
 - If the user requested analysis for a specific function, the traverser searches for a `function_definition` or `method_declaration` node whose identifier matches the requested name.
 - If a whole-file analysis is requested, the system instead extracts all function names and their direct dependencies (functions they call) to build the `func_dep_graph`.
4. **Line-by-Line Logic Extraction:** Once inside a target function, a recursive `process_stmt()` function kicks in. This function acts as a massive switch statement against Tree-sitter node types:
 - **Assignments/Expressions:** Handled as a `process` node (e.g., `x = 5`).
 - **Control Flow (`if`, `while`, `for`):** Handled as a `decision` or `loop` node. The condition is extracted as the node label. Crucially, the traverser then recursively executes `process_stmt()` on the `consequence` (True branch) and the `alternative` (False branch), wiring edges accordingly.
 - **Return Statements:** Handled as a `terminator` node with a red color.
 - **External Calls:** Tracked when the code invokes a function imported from another file.
5. **JSON Payload Generation:** The `ReactFlowBuilder` returns a dictionary: `{"nodes": [...], "edges": [...]}`.

3. Cyclomatic Complexity Calculation

To help developers identify "technical debt," CodeMap calculates the Cyclomatic Complexity for every analyzed function.

Python

For Python, CodeMap leverages the `radon` library. Radon visits the AST and calculates complexity based on Thomas J. McCabe's strict cyclomatic complexity rules (counting control flow branches).

Other Languages (JS, Java, C++)

Since `radon` is Python-only, the backend implements a custom heuristic complexity calculator for tree-sitter ASTs.

The `calculate_js_complexity(code)`, `calculate_java_complexity(code)`, and `calculate_cpp_complexity(code)` functions work by:

1. Parsing the code using Tree-sitter.
2. Initializing a base score of 1.
3. Traversing the AST and incrementing the score by 1 every time it encounters branching logic nodes specific to that language's grammar, such as:

- `if_statement`
- `for_statement`
- `while_statement`
- `catch_clause`
- `ternary_expression`
- `&&` and `||` logical operators.

Utilization in the Frontend

This complexity score serves two purposes:

1. **The Heatmap:** In the frontend's `FuncDepGraph`, nodes are colored based on this score. Scores < 5 are green, 5-10 are yellow, > 10 are red.
2. **The Inspector:** The right-hand panel sorts all files in the repository by their highest-complexity function, allowing developers to instantly zero in on the most convoluted areas of the codebase.

CodeMap: Git Integration, File Management & Data Export

Overview

CodeMap is designed to handle multiple sources of codebase input seamlessly, abstracting away the underlying file system complexities so developers can focus purely on architecture and analysis.

This document details the precise mechanics behind the three core data pipelines: **Local Project Explorer**, **GitHub Repo Explorer**, and **Image Exporting**.

1. Local Project Explorer: The File Upload Pipeline

Users can upload single files, complete directories, or raw `.zip` archives directly into CodeMap's frontend.

The Mechanism

1. **Frontend Dropzone:** The frontend uses the HTML5 `<input type="file" multiple webkitdirectory />` API to natively handle local folder drops.
2. **Reading the Contents:** The `FileReader` API asynchronously converts the dropped files into text strings.
3. **The FileExplorer Component:**
 - Instead of presenting a raw list, CodeMap builds an internal hierarchical tree named `fileTree` matching the physical directory structure. Iterating over the file paths, it creates nested objects: `{ name, path, type: 'folder'|'file', children: {} }`.
4. **Sending to Backend:**
 - Single files are sent directly to the `/analyze` endpoint via a standard HTTP POST request.
 - Entire directories are sent to a specialized `/analyze-multi` endpoint. The frontend maps the `fileTree` into a flat array of `{ filename: string, content: string }` dictionaries and sends the entire codebase in a single JSON payload.
5. **Multi-File Caching (crossFileData):**
 - The backend specifically parses imports and maps them across the payload. If `main.py` imports `utils.py`, the backend explicitly creates a metadata dictionary linking the function calls.
 - The frontend caches this payload inside the root `App_v2.js` component payload, allowing for rapid navigation between files without requiring a backend round-trip every single click.

2. GitHub Integration: The Remote Repository Pipeline

CodeMap allows dynamic exploration of public GitHub repositories without requiring a local `git clone`.

Architecture & APIs

1. **Repo URL Parsing:** The frontend takes the standard `github.com/user/repo` format and transforms it into an API request: `https://api.github.com/repos/user/repo`.
2. **Recursive Tree Fetching:** CodeMap avoids downloading massive `.zip` files from GitHub. Instead, it hits the Git REST API endpoint `/git/trees/{branch}?recursive=1`. This returns a massive JSON manifest of every single file path and its sha hash inside the repository.
3. **Frontend Tree Construction:** The frontend takes this flat array of file paths and dynamically constructs a hierarchical folder tree structure, identical in format to the local `FileExplorer`.
4. **Lazy Content Loading:** CodeMap does **not** download the repository's source code upfront. It waits. When a user clicks a file node in the sidebar tree, the frontend triggers a targeted `axios` GET request to `https://raw.githubusercontent.com/user/repo/branch/filepath` to fetch specifically that file's raw text.
5. **Backend Analysis Offloading:** To generate a bird's-eye view of the entire repository, CodeMap sends an HTTP POST to the backend's `/analyze-project` endpoint, passing the GitHub repository URL.
 - The Python backend uses the `requests` library to fetch the zipball from GitHub natively.
 - It extracts it in-memory, recursively traverses the filesystem, and aggregates Cyclomatic Complexity metrics for every function in every supported file.
 - It returns a massive JSON payload mapping `file_path -> { functions: [], complexity: {...} }`. The frontend stores this strictly in a state variable `githubBlueprintData` to populate the Heatmap and Inspector panels.
6. **Recent Commits Polling:** As users navigate the repo map, CodeMap silently polls `https://api.github.com/repos/user/repo/commits?path=filepath` in the background, updating the sidebar to show exactly who last modified the actively viewed file.

3. High-Fidelity Image Exporting

A crucial feature of CodeMap is its ability to generate sharing-ready snapshots of the interactive React Flow canvas.

The Mechanism

1. **The Tooling:** CodeMap relies on the `html-to-image` open-source library.
2. **DOM Traversal:** The frontend triggers the `toPng()` function against the specific HTML ref of the `.react-flow__renderer` DOM element.
3. **SVG Rendering Pipeline:**
 - `html-to-image` walks down the DOM tree of that specific element, cloning every `<div>`, `<svg>`, `<canvas>`, and CSS style block locally.
 - It serializes this massive internal DOM tree into an SVG string.
 - Using an HTML5 `<canvas>` element drawn temporarily in browser memory, it renders the SVG string using a `data:image/svg+xml` URI.
4. **Data URI Conversion:** The resulting canvas data is extracted as a base64 encoded `.png` string.
5. **Download Trigger:** A temporary hidden `<a>` HTML tag is injected into the DOM with its `href` set to the generated base64 string and its `download` attribute set to `codemap-graph.png`. The frontend simulates a click on this invisible link, initiating the browser's native file download prompt.

CodeMap: UI Architecture & Interactive Features

Overview

CodeMap provides a seamless, VS Code-like interface designed to display massive hierarchical data structures (code architecture) interactively without overwhelming the user. Built entirely in React 18, the frontend revolves around specialized components managing global layouts, graph canvas interactions, and multi-layered sidebars.

This document breaks down the frontend architecture and how specific UI features operate.

1. Global Layout: Panes & Conditional Rendering

The main wrapper `App_v2.js` serves as the centralized state manager and container. The application is divided into three primary horizontal flex areas:

1. **The Navigation Sidebar (Left):** Houses the Activity Bar and the primary tools (File Explorer or GitHub Repo input).
2. **The Graph Canvas (Center):** The dominant area dedicated to rendering `React Flow` components (`FileDepGraph`,

FuncDepGraph, FlowGraph).

3. **The Data Inspector (Right):** A specialized context-sensitive panel explaining the active graph (function stats, github repo files, etc.).

Collapsible Paneling

A core UX principle is preserving "Screen Real Estate" for the main graph canvas.

- The left sidebar (`<FileExplorer />` or `<GitHubExplorer />`) and the right `Inspector` panel each maintain dedicated collapsed state variables (`sidebarCollapsed`, `rightPanelCollapsed`).
- When a user clicks the respective chevron toggle (`<` or `>`), the React layout engine conditionally removes the child components and resizes the container `div` width from `320px` down to `32px`.
- The panels are joined by customized `Draggable` dividers that allow users to seamlessly drag-to-resize the panel widths interactively.

2. Interactive Graphing Engine

CodeMap replaces traditional static images with highly interactive node structures rendered via the `reactflow` library.

Automatic Node Layout (Dagre)

CodeMap does not calculate raw `x`, `y` coordinates manually on the backend. This would break dynamically sized screens. Instead:

- The backend sends a purely topological graph `{"nodes": [], "edges": []}` with no fixed positions.
- The React frontend ingests this and feeds it into the `dagre` (Directed Graph) layout algorithm.
- `Dagre` mathematically calculates the most efficient, non-colliding top-down layout mapping and injects `{x, y}` positioning data into every node before passing the final object cleanly into the `ReactFlow` renderer.

Graph Types

1. **FileDepGraph:** Maps how individual files depend on other files (`import` and `require` chains).
2. **FuncDepGraph:** Maps how individual functions *inside* a selected file depend on each other.
3. **FlowGraph:** The detailed inner-logic flowchart of a specific function.

Node Types

CodeMap registers highly specialized custom React components to replace default flow nodes to match specific programming constructs:

- **process:** Basic assignments (`x = 5`).
- **decision:** Control statements like `if/else` rendered as dynamic orange diamonds.
- **loop:** Constructs like `for` and `while` loops.
- **terminator:** `Return` objects rendered natively in Red, helping intuitively map function exits.
- **externalCall:** Function invocations to other files rendered natively in Purple.

3. High-Fidelity UI Features

CodeMap wraps the graph canvas with several advanced UI tools:

Global Glassmorphism Search Bar

- To allow users to instantly find functions inside massive 500+ node graphs, an absolute-positioned floating search bar sits in the top right.
- As the user types, the `onChange` event directly modifies the `ReactFlow` node array. If a node label doesn't string match the search input (case insensitive), its `style.opacity` drops to 12% (0.12).
- Crucially, connected edges remain bright, acting as a visual spotlight instantly illuminating architectural funnels.

Technical Debt Visualizer (The Heatmap)

- CodeMap utilizes a "🔥 Heatmap" toggle next to the global search.
- When toggled on, the backend's provided Cyclomatic Complexity score (`node.data.complexity`) hooks into a dynamic color mapping function (`getHeatmapColor()`).
- The standard, flat language-based colors are entirely bypassed:
 - Complexity < 5: **Green**
 - Complexity 5-10: **Yellow/Orange**
 - Complexity > 10: **Crimson Red**
- This instantly turns an abstract diagram into a live technical debt triage map.

Live Minimap Observer

- A React Flow `MiniMap` component operates in the bottom-right corner of the graph canvas.
- It maintains a live 1:1 reflection of the main canvas, intercepting the `nodeColor` property logic so that the tiny indicator map accurately reflects the Red/Green/Grey structure of the user's current zoom level.